

IT Projektarbeit – Gruppe 31

on&on

Deployment manual

Ausgeführt von:

Kostenzer, Cosima, if25b187

Rank, Leonie, if25x038

Schober, Max, if25b025

Stutterecker, Sinja, if25b043

Betreuer*in: Lena Esther Oswald

Wien, 19. Juni 2026

Inhaltsverzeichnis

1. Introduction.....	3
2. System Requirements.....	3
3. Application Deployment.....	4
3.1. Clone Application Files.....	4
3.2. Install Dependencies.....	4
3.3. Environment Configuration.....	4
3.4. Generation of Application Key.....	5
3.5. Configure File Permissions.....	5
3.6. Build Frontend Assets.....	5
3.7. Create Storage Symlink.....	5
4. Database Initialization.....	6
4.1. Run Database Migrations.....	6
4.2. Seed Initial Data.....	6
5. Caching.....	6
6. External Services.....	7
7. Alternative Deployment Solutions.....	7
8. Sources and additional information.....	7

1. Introduction

This document describes the deployment process for onzon, an outfit recommendation system developed with the Laravel framework. During development, the application was executed locally using Laravel Herd or Docker, which provide isolated environments with all required dependencies. For production deployment, the application must be hosted on a publicly accessible web server and connected to a production database.

Laravel applications are best hosted on Linux-based servers due to their stability, security, and compatibility with the PHP ecosystem. In addition to a web server, a registered domain name (URL) is required to make the application publicly accessible.

The application also relies on external weather data provided by the Open-Meteo API. This service is used to retrieve current weather information for the selected location and to generate appropriate outfit recommendations. Additionally the geocoding service Nominatim is used to transfer city names into latitude and longitude or reversed if the current location is used.

2. System Requirements

The target server must satisfy the following software requirements:

PHP Requirements

- PHP >= 8.2
- CType PHP Extension
- cURL PHP Extension
- DOM PHP Extension
- Fileinfo PHP Extension
- Filter PHP Extension
- Hash PHP Extension
- Mbstring PHP Extension
- OpenSSL PHP Extension
- PCRE PHP Extension
- PDO PHP Extension
- Session PHP Extension
- Tokenizer PHP Extension
- XML PHP Extension

Additional Requirements

- Linux-based operating system (recommended)

- Web server such as Nginx or Apache
- Composer dependency manager
- Relational database system (MySQL or MariaDB)
- Version control system Git
- Node.js and NPM for frontend assets compilation
- Internet access for Open-Meteo and Nominatim API communication

The web server user must have write permissions for the following directories:

bootstrap/cache
storage

These permissions are required for Laravel's caching, logging, and file storage functionality.

3. Application Deployment

3.1. Clone Application Files

The Laravel project files must be cloned to the server in an appropriate directory (for example /var/www on ubuntu) using Git clone command:

```
git clone https://github.com/itproject-onion/onion.git
```

3.2. Install Dependencies

Navigate to the project directory and install the required PHP dependencies:

```
composer install --no-dev --optimize-autoloader
```

3.3. Environment Configuration

Before deploying the application, the environment configuration file .env must be adapted for the production environment.

Database Configuration

The database credentials must be updated to match the production database server:

```
DB_CONNECTION=mysql  
DB_HOST=localhost  
DB_PORT=3306
```

```
DB_DATABASE=database_name
DB_USERNAME=username
DB_PASSWORD=password
```

Application Settings

```
APP_ENV=production
APP_DEBUG=false
```

Debug mode must be disabled in production environments to prevent sensitive application information from being exposed to end users when unexpected exceptions occur.

3.4. Generation of Application Key

If no application key exists, generate one using:

```
php artisan key:generate
```

3.5. Configure File Permissions

Ensure that the web server has write access to the required directories:

```
chmod -R 775 storage bootstrap/cache
```

3.6. Build Frontend Assets

Run the following commands to install all frontend dependencies and generate optimized production assets that are served by the web application.

```
npm install
npm run build
```

3.7. Create Storage Symlink

Laravel stores publicly accessible uploaded files in the storage directory. To make these files available through the web server, it is necessary to create a symbolic link with the following command:

```
php artisan storage:link
```

4. Database Initialization

After the application has been deployed and the database connection has been configured, the database schema must be created.

4.1. Run Database Migrations

The following command creates all required database tables:

```
php artisan migrate
```

4.2. Seed Initial Data

The application requires predefined categories and default clothing items. These records are inserted using Laravel seeders:

```
php artisan db:seed --class=CategorySeeder
```

```
php artisan db:seed --class=ItemSeeder
```

After these commands have been executed successfully, the database contains all required initial data and the application is ready for use.

5. Caching

Caching is highly recommended for production deployments. To improve performance, generate Laravel's cache files using the following commands:

```
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

If changes are made later to configuration, routes or views, the caches should be cleared and rebuilt:

```
php artisan optimize:clear
```

```
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

6. External Services

Open-Meteo Weather API

The application uses the Open-Meteo Weather API to retrieve current weather conditions for user-selected locations.

The weather data is processed by the recommendation system to generate suitable outfit suggestions. Therefore, the deployment environment must allow outgoing HTTPS requests to external services.

If the API is unavailable, an error is shown and the application can not be used.

Nominatim Geocoding Service API

The application uses the Nominatim Geocoding Service API to transfer city names into latitude and longitude when searching for a location or to look up the name of the place if the current location is used.

The Open-Meteo Weather API is then used to retrieve the weather for the given latitude and longitude. Therefore outgoing HTTPS requests are required for it, too.

If the API is unavailable, it is not possible to search for a city, however the current location will be used without showing the name of the place.

7. Alternative Deployment Solutions

Instead of manually managing a server, Laravel applications can also be deployed using managed Laravel hosting solutions like Laravel Cloud or Laravel Forge.

These platforms provide many tools out of the box for example infrastructure management and deployment automation which reduce operational effort and make deployment much easier.

8. Sources and additional information

More information about deployment of Laravel applications can be found here:

<https://laravel.com/docs/12.x/deployment>

<https://laraspeed.dev/blog/deploy-laravel-production-guide>

<https://cloud.laravel.com/docs/intro>

<https://forge.laravel.com/docs/sites/laravel>